Logistic Inference: Confidence Intervals

Dr. Josh Jacobson

27 February 2026

These lecture notes were originally developed by Dr. Kris Pruitt.

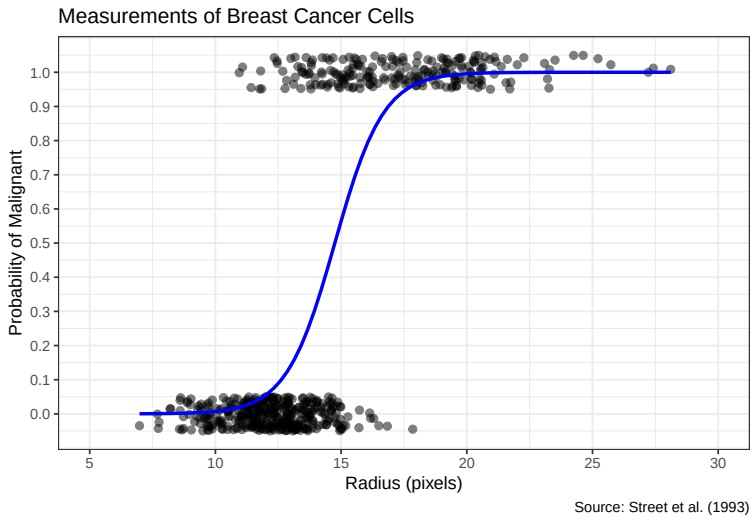
The purpose of this lesson is to demonstrate computational and mathematical methods for constructing confidence intervals for the slope parameter of a simple logistic regression model.

Computational Confidence Interval

We will once again employ the breast cancer study from the University of Wisconsin to demonstrate the methods of this lesson. Import, wrangle, and visualize the data below.

```r
# import and wrangle data
cancer <- read_csv("cancer.csv") %>%
  transmute(
    Diagnosis = ifelse(diagnosis == "M", 1, 0),
    Radius = radius_mean
  ) %>%
  na.omit()

# create scatter plot
ggplot(data = cancer, aes(x = Radius, y = Diagnosis)) +
  geom_jitter(height = 0.05, size = 2, alpha = 0.5) +
  geom_smooth(
    formula = y ~ x, method = "glm",
    method.args = list(family = "binomial"), se = FALSE, color = "blue"
  ) +
  labs(
    title = "Measurements of Breast Cancer Cells",
    x = "Radius (pixels)",
    y = "Probability of Malignant",
    caption = "Source: Street et al. (1993)"
  ) +
  scale_y_continuous(limits = c(-0.05, 1.05), breaks = seq(0, 1, 0.1)) +
  scale_x_continuous(limits = c(5, 30), breaks = seq(5, 30, 5)) +
  theme_bw()
```



The visualization suggests a relatively strong, positive association between the radius of a cell and its diagnosis as malignant or benign. The estimated parameters of the logistic function are determined by the maximum likelihood method below.

```
# fit logistic regression model
model <- glm(Diagnosis ~ Radius, data = cancer, family = "binomial")

# view estimates coefficients
coefficients(summary(model))
```

```
##               Estimate Std. Error   z value     Pr(>|z|)
## (Intercept) -15.245871 1.32462600 -11.50957 1.180708e-30
## Radius        1.033589 0.09310645  11.10115 1.238377e-28
```

Based on the summary output, the equation for the best-fit log-odds function is shown below.

$$\log\left(\frac{P(\text{Malignant}|\text{Radius}=x)}{P(\text{Benign}|\text{Radius}=x)}\right) = 1.034x - 15.246$$

In the previous lesson, we conducted hypothesis tests to evaluate evidence that the true slope parameter of this equation differs from zero. The tests offered very strong evidence that the slope parameter is *not* zero. Yet, such tests do not provide an estimate of what the slope could be. For that we need a confidence interval.

As with other inferential techniques, we can approach confidence intervals from either a computational or mathematical viewpoint. Computationally, we can leverage bootstrap resampling to create the interval.

Activity

Construct a 99% bootstrap confidence interval for the slope coefficient on cell radius.

```
# initiate empty vector
results2 <- data.frame(beta1 = rep(NA, 2000))

# repeat bootstrap process 2000 times and save results
for (i in 1:2000) {
  set.seed(i)
  boot <- cancer[sample(nrow(cancer), replace = TRUE), ]

  # Step 1: Use bootstrap resampling to draw n samples from the data
  # distribution, where n is the number of rows in the cancer dataframe;
```

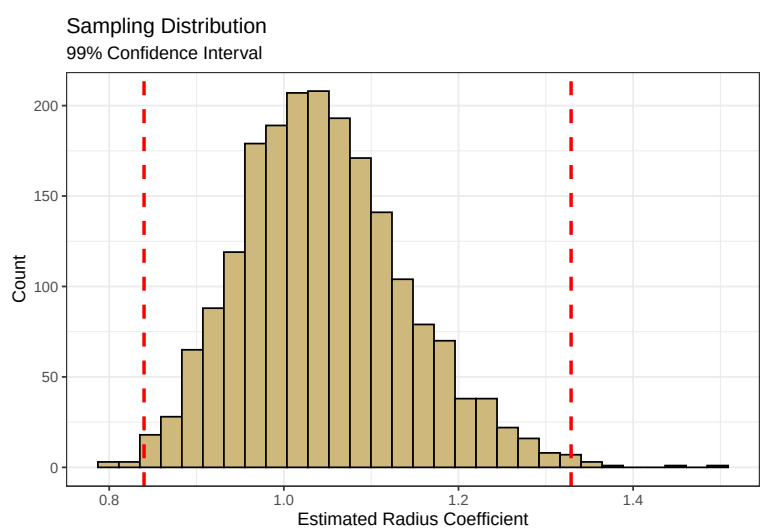
```r
  # call the result "boot"
  # Note: do we sample with replacement here?

  # Step 2: Fit a logistic regression model using the boot data;
  # call it "remodel2"
  remodel2 <- glm(Diagnosis ~ Radius, data = boot, family = 'binomial')
  results2$beta1[i] <- coefficients(summary(remodel2))[2, 1]
}
```

We visualize the sampling distribution and rejection region using a histogram.

```r
# compute confidence bounds
threshold1 <- as.numeric(quantile(results2$beta1, 0.005))
threshold2 <- as.numeric(quantile(results2$beta1, 0.995))

# plot sampling distribution
ggplot(data = results2, aes(x = beta1)) +
  geom_histogram(color = "black", fill = "#CFB87C") +
  geom_vline(
    xintercept = threshold1, color = "red",
    linewidth = 1, linetype = "dashed"
  ) +
  geom_vline(
    xintercept = threshold2, color = "red",
    linewidth = 1, linetype = "dashed"
  ) +
  labs(
    title = "Sampling Distribution",
    subtitle = "99% Confidence Interval",
    x = "Estimated Radius Coefficient",
    y = "Count"
  ) +
  theme_bw()
```



As expected, the sampling distribution is centered on the point estimate of $\hat{\beta}_1 = 1.034$. However, some samples produced slopes as high as 1.5 and as low as 0.8. The 99% confidence bounds are 0.840 and 1.329. Thus, we are 99% confident that the true slope of the log-odds function is between 0.840 and 1.329. Said another way, for every one pixel increase in cell radius we are 99% confident that the log-odds of being

malignant will increase by between 0.840 and 1.329. Although log-odds are not as intuitive, they do increase with probability.

Mathematical Confidence Interval

By this point in the course, it should be clear that the primary difference between computational and mathematical approaches to inference regard the characteristics of the appropriate distribution. For hypothesis tests we need a null distribution, while for confidence intervals we require a sampling distribution. Computational approaches simulate these distributions directly from the data while mathematical approaches assume named distributions based on theory. In this specific case, theory suggests the estimated slope of a log-odds function follows the Normal distribution. As a result, we can construct a $100 \cdot (1 - \alpha)\%$ confidence interval using the following formula:

$$\hat{\beta}_1 \pm z_{\alpha/2} \cdot SE_{\hat{\beta}_1}$$

The first two components of the confidence interval formula are relatively straight-forward. We already have a point estimate of $\hat{\beta}_1 = 1.034$. At the 99% level of confidence, the critical value $z_{0.005}$ is computed from the standard Normal distribution.

```
# compute critical value
qnorm(p = 0.005, mean = 0, sd = 1, lower.tail = FALSE)
```

```
## [1] 2.575829
```

In order to achieve 99% confidence, we must include 2.58 standard errors above and below the point estimate. The final piece is to determine the value of the standard error $SE_{\hat{\beta}_1}$ itself. We have multiple options for obtaining an estimate of the standard error. The simplest option is to extract it directly from the summary output (0.0931), but we discuss two other options below.

Another method for estimating the standard error of $\hat{\beta}_1$ is to calculate it from the bootstrapped point estimates in the previous section. The `results2` vector holds 2,000 sample slopes and the standard deviation of those values is a good estimate for $SE_{\hat{\beta}_1}$.

```
# estimate standard error
sd(results2$beta1)
```

```
## [1] 0.09498488
```

Our bootstrapped estimate of 0.0950 does differ slightly from the value of 0.0931 in the summary output. But it is not dramatically different. If we want to fully reproduce the value in the summary output, then we must compute and invert the **Fisher Information Matrix** referenced in the previous lesson. Here we will dig a bit deeper into its derivation. Recall that the log-likelihood function for a simple logistic regression model is:

$$l(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^{n} [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

where,

$$\hat{p}_i = \frac{1}{1 + e^{-(\hat{\beta}_0 + \hat{\beta}_1 x_i)}}$$

It turns out that the Fisher Information Matrix, $\mathbf{I}(\hat{\beta})$, is equivalent to the negative expected Hessian matrix of this log-likelihood function. The Hessian is the matrix of second-order partial derivatives with respect to $\hat{\beta}_0$ and $\hat{\beta}_1$. As a multivariate calculus refresher, you should confirm that the negative expected Hessian is:

$$\mathbf{I}(\hat{\beta}) = \begin{bmatrix} \sum \hat{p}_i(1-\hat{p}_i) & \sum x_i\hat{p}_i(1-\hat{p}_i) \\ \sum x_i\hat{p}_i(1-\hat{p}_i) & \sum x_i^2\hat{p}_i(1-\hat{p}_i) \end{bmatrix}$$

Three days later... As complex as this may appear, we can compute the components of this matrix relatively easily from the training data.

```
# compute Fisher components
cancer %>%
  mutate(p_hat = predict(model, newdata = cancer, type = "response")) %>%
  summarize(
    comp11 = sum(p_hat * (1 - p_hat)),
    comp12 = sum(Radius * p_hat * (1 - p_hat)),
    comp22 = sum(Radius^2 * p_hat * (1 - p_hat))
  )
```

```
## # A tibble: 1 x 3
##   comp11 comp12 comp22
##    <dbl>  <dbl>  <dbl>
## 1   51.0   722. 10332.
```

Based on the summary, our Fisher Information Matrix is:

$$\mathbf{I}(\hat{\beta}) = \begin{bmatrix} 51.04 & 722.15 \\ 722.149 & 10331.84 \end{bmatrix}$$

Now we have arrived at the most important step. The inverse of the Fisher Information Matrix is equivalent to the covariance matrix for $\hat{\beta}_0$ and $\hat{\beta}_1$. We can invert the matrix with the following code.

```
# create Fisher matrix
fisher <- matrix(data = c(51.04474, 722.1487, 722.1487, 10331.84), nrow = 2, ncol = 2)

# invert Fisher matrix
solve(fisher)
```

```
##            [,1]         [,2]
## [1,]  1.7549245 -0.122661253
## [2,] -0.1226613  0.008670253
```

This should look familiar from the previous lesson. It is equivalent to the covariance matrix we extracted from the summary output. As before, the lower diagonal element is the variance of $\hat{\beta}_1$ and the square root is the standard error. After all of that, we find that $SE_{\hat{\beta}_1} = \sqrt{0.00867} = 0.0931$ and we can finally produce our confidence interval.

$$1.034 \pm 2.58 \cdot 0.0931$$
$$1.034 \pm 0.240$$

We are 99% confident that the true slope of the log-odds function is between 0.794 and 1.274.